

HDFC Form Filler — `content.js` Technical Documentation

Overview

`content.js` is a Chrome extension **content script** that runs in the context of HDFC AEM form pages. It has two primary modes:

1. **Recording** — captures every field fill and button click the user makes into a step sequence
2. **Replay** — replays that sequence automatically, handling async UI (AEM visibility rules, API-loaded dropdowns, same-tab redirects)

Architecture

```
Page Load
├── interceptNetwork()      [IIFE – patches fetch + XHR globally]
├── chrome.storage allowedPatterns check
├── activate()
│   ├── restoreRecordingStateSync() [sync – before listeners]
│   ├── FORM_READY → background    [async – check redirect resume]
│   └── attach DOM listeners
│       ├── input/change → handleInputChange (recordedData)
│       ├── change       → handleStepChange  (recordedSteps)
│       └── click         → handleStepClick   (recordedSteps + redirect
detection)
```

Modules

1. Network Intercept

Monkey-patches `window.fetch` and `XMLHttpRequest.prototype.send` to maintain an `activeRequests` counter. `waitForNetwork()` polls this counter and resolves when the page is idle. Used throughout replay to ensure each action's side-effects (API calls triggered by field changes) complete before the next step.

2. Activation Guard

Reads `allowedPatterns` from `chrome.storage.local`. If configured, only activates on matching URLs. If empty, activates everywhere. All listener attachment and replay logic is gated behind this.

3. Recording State (`recordedData` + `recordedSteps`)

Two parallel stores:

Store	What it holds	Used for
recordedData	{ fieldName: value } snapshot	Simple prefill replay (START_REPLAY)
recordedSteps	Ordered [{type, name, value} {type, tag, text}]	Step-by-step replay (START_STEP_REPLAY)

Persistence: Both are serialised to `localStorage` under key `__hdfc_ext_recording__` after every change via `persistRecordingState()`. This allows state to survive same-tab navigation (e.g. a Perfios redirect that replaces the page).

Restore logic (`restoreRecordingStateSync`): Called synchronously at the top of `activate()`, before any listeners are attached (prevents the restored steps from being re-recorded as new user input).

- If `performance.navigation.type === 'reload'` → **clears** `localStorage` (reload = fresh start)
- If navigated (redirect) → restores `recordedData` and `recordedSteps` into memory
- State older than 2 hours is discarded

4. Step Recording

`handleStepChange` — fires on `change` events:

- Skips hidden fields, OTP fields, readonly/disabled inputs, and fields without a name
- For `<select>`, also captures the display label (for typeahead replay)
- Deduplicates consecutive identical fills for the same field

`handleStepClick` — fires on `click` events:

- Records { type: 'click', tag, text, name, index } for buttons/submits
- After recording, registers a `beforeunload` listener to detect if this click caused navigation:
 - Same URL → marks step `expectsReload: true`, clears `localStorage`
 - Different URL → marks step `expectsRedirect: true`
- Only the **last** click before navigation is marked (previous pending listeners are cancelled to avoid multi-flag bugs)

5. Prefill Replay (`START_REPLAY` mode)

A simple, non-sequential fill mode triggered by the popup:

- `prefillVisibleFields()` iterates all `input/select/textarea` elements, fills any that appear in `replayData` and are currently visible
- A `MutationObserver` watches for `data-visible` attribute changes on the AEM form — when a new section becomes visible, `prefillVisibleFields()` is called again automatically
- `fillInProgress` + `pendingFill` flags prevent concurrent fill loops

6. Step Replay (`START_STEP_REPLAY` mode)

The main replay engine — `replaySteps(steps, stopAfterIndex)` :

Iterates the recorded steps array sequentially. For each step:

Fill steps:

Input type	Strategy
radio	Finds the specific [value=X] radio, clicks its label, sets .checked, dispatches events
checkbox	Clicks label, sets .checked, dispatches events
Native <select>	focus → mousedown → click (to trigger lazy option loading) → waitForSelectOption → sets .value
Typeahead <select>	Detects hidden <select> + visible custom input → typeCharByChar → waitForDropdownOption → clicks overlay option
range (slider)	Sets .value → dispatches input + mouseup + change (AEM uses mouseup for rule evaluation)
Text / number / date	Clears existing value → sets new value → dispatches input / change / blur

Click steps:

1. `waitForClickable()` — polls until element is in DOM, non-disabled, and has layout dimensions
2. Sends `SAVE_RESUME_STATE` to background before firing (in case the click causes navigation)
3. Three click outcomes handled:
 - **Opens new tab:** waits for `RESUME_AFTER_TAB_CLOSE` message from background, then continues
 - **Same-tab redirect** (`expectsRedirect: true`): fires click, waits for `beforeunload` , suspends — background's `FORM_READY` resumes on the new page
 - **Normal click:** fires, checks if button disappeared/was disabled; retries if button is still present (form validation blocked it); advances if next step's element is now visible (modal opened)

`waitForFillable(name)` : Polls until `[name=X]` is in the DOM and not AEM-hidden. Falls back after 30s to handle edge cases where visibility rules never fire.

`waitForSelectOption(selectEl, value)` : Polls until the target option appears in the `<select>` (handles API-loaded dropdowns). Falls back to the first real option after 5s.

7. Cross-Page Redirect Resume

The most complex flow — when a button navigates the same tab to a different origin (e.g. a bank verification redirect):

Content script (page A)

- └ Before click: `SAVE_RESUME_STATE` → background (`chrome.storage.session`)
- └ Fires click → `beforeunload` → content script destroyed

Intermediate page (different origin)

- └ `FORM_READY` → background returns `{ resume: false, pendingState: true }`
- └ Content script does nothing, preserves `localStorage`

Original form page (page A, reloaded after redirect back)

- └ `FORM_READY` → background returns `{ resume: true, steps, resumeFromStep, stopAfterIndex }`

(click: registers beforeunload → marks expectsRedirect/Reload)

persistRecordingState()

localStorage['__hdfc_ext_recording__']
{ data, steps, pathname, savedAt }

Prefill Replay Flow (START_REPLAY)

Popup

└─ sendMessage(START_REPLAY, { data })

content script sets replayData

└─ prefillVisibleFields()

└─ for each visible input: set .value → dispatchEvents()

AEM rule engine re-evaluates
(may show/hide other sections)

└─ MutationObserver watches data-visible attributes

└─ section becomes visible → prefillVisibleFields() again
(loop continues until all visible fields are filled)

Step Replay Flow (START_STEP_REPLAY)

Popup (or FORM_READY redirect resume)

└─ sendMessage(START_STEP_REPLAY, { steps, stopAfterIndex })

replaySteps(steps)

└─ [fill step]

└─ waitForFillable(name) ← polls DOM until field visible (max 30s)

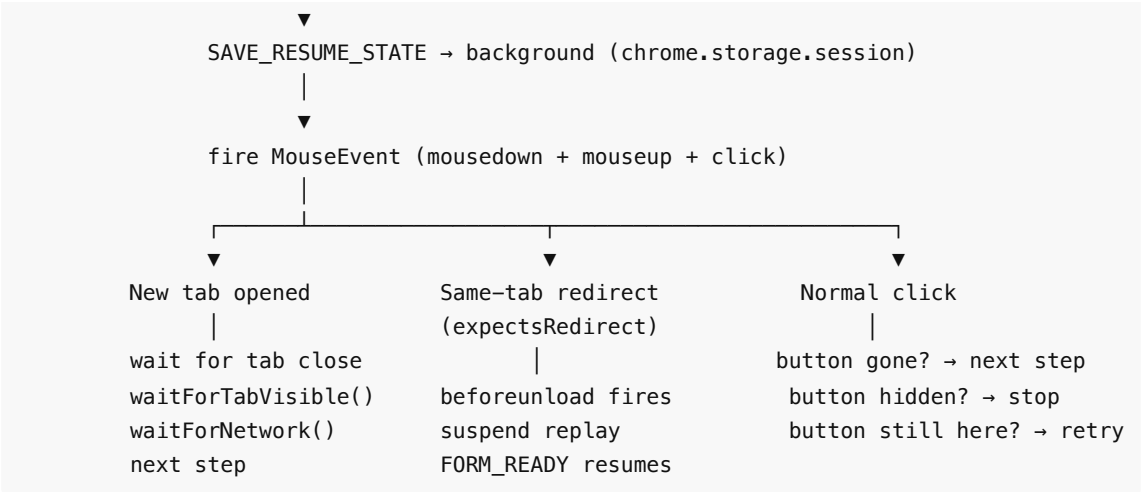
set value / dispatch events

waitForNetwork() ← waits for all fetch/XHR to complete

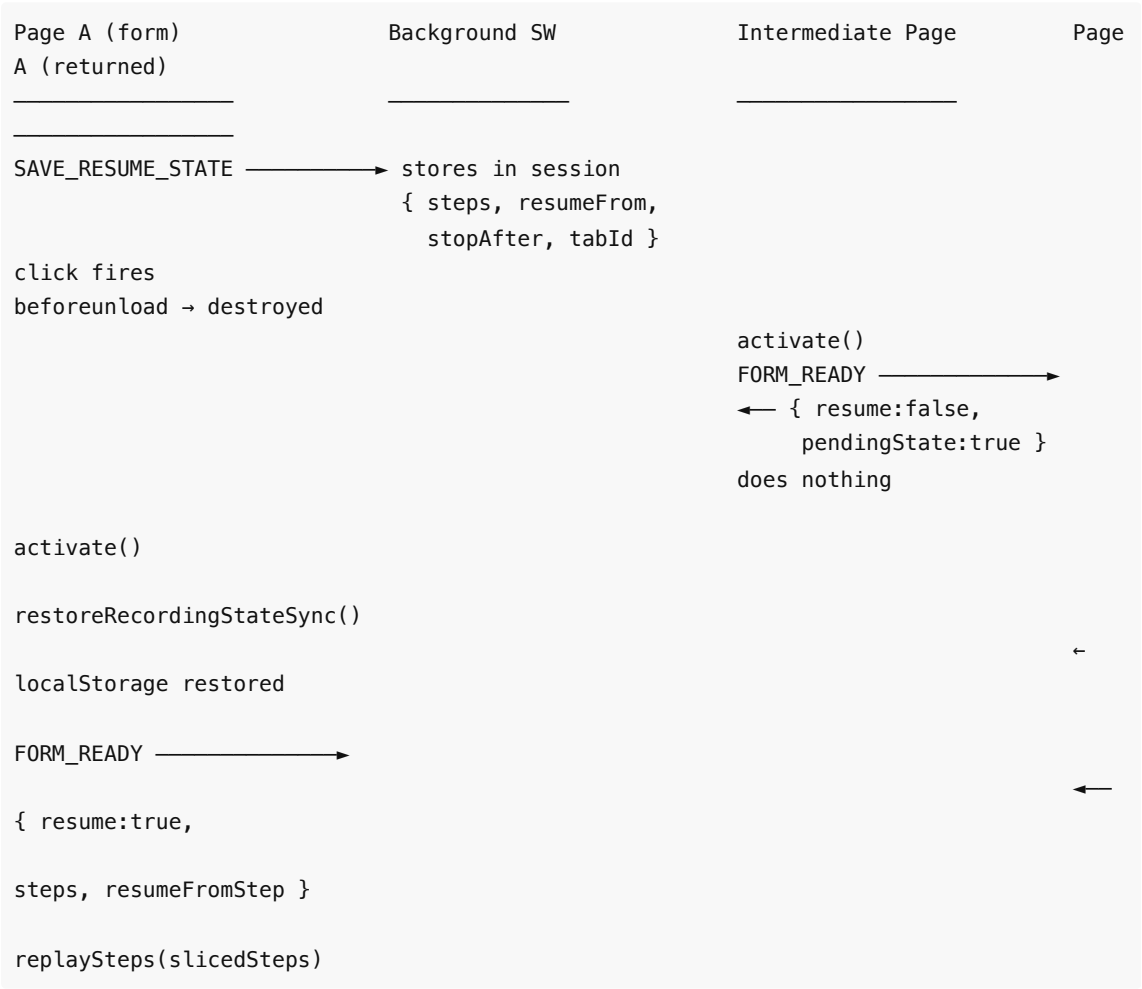
next step

└─ [click step]

└─ waitForClickable(step) ← polls until visible + non-disabled



Cross-Page Redirect State Flow



Storage Layers

Store	What	Survives
-------	------	----------

<code>localStorage['__hdfc_ext_recording__']</code>	<code>recordedData + recordedSteps</code>	Same-origin navigation
<code>chrome.storage.session (background)</code>	Replay cursor (steps, <code>resumeFromStep</code> , <code>stopAfterIndex</code>)	SW restarts, cross-origin redirects

What the Extension Cannot Do

Hard Limitations (by design)

- **OTP fields** — explicitly excluded by name pattern; OTP must always be entered manually
- **CAPTCHA** — no solving capability; any CAPTCHA in the journey will block replay
- **File upload inputs** — `<input type="file">` is not handled; file selection cannot be scripted via DOM events
- **Cross-origin iframes** — content script runs in the top frame only; fields inside cross-origin iframes are invisible to it
- **Multi-tab replay** — when a click opens a new tab, the extension only waits for that tab to close; it does not record or replay inside the new tab

Field-Type Limitations

- **Readonly / disabled fields** — skipped entirely; if a field is programmatically locked, it will not be filled
- **Fields with unstable name attributes** — replay matches fields by `name` ; if the AEM form regenerates names between sessions, replay breaks silently
- **Custom date/time pickers (non-native)** — only `<input type="date/time">` is handled; calendar widget clicks are not recorded
- **Drag-and-drop interactions** — not recorded or replayed
- **Signature pads / canvas inputs** — no canvas interaction support
- **Multi-select dropdowns** — only single-value selects are handled

Replay Behaviour Limits

- **No rollback / undo** — if a step fails mid-replay, the form is left in a partially filled state with no recovery
- **No branching logic** — replay follows the exact recorded path; if the form presents a different branch (e.g. different loan product), replay will fill wrong fields or stall
- **Sequential only** — steps execute one at a time; long forms with many API calls are slow
- **Stop-after is one-directional** — `stopAfterIndex` can pause at a step but cannot start replay from an arbitrary middle step without a prior `SAVE_RESUME_STATE`
- **Typeahead fallback is best-effort** — if neither the overlay option nor the native `<select>` option matches within 5s, the first non-empty option is selected as a fallback, which may be wrong
- **State TTL is 2 hours** — recording state in `localStorage` expires after 2 hours; longer sessions lose their restore point

Infrastructure Limits

- **Single tab only** — the redirect resume mechanism is keyed by `tabId` ; if the user opens the form in multiple tabs simultaneously, state from one tab can interfere with another
- **Allowed URL patterns** — if `allowedPatterns` is configured in extension storage and the current URL does not match, the content script does not activate at all